CST8507: Natural Language Processing

Lab 3: Word Embedding

Objective

This assignment will provide you with experience with the following:

- Load pre-trained word vectors.
- Evaluate embeddings using intrinsic metrics
- Use word embeddings to solve word analogy problems.
- Identify how word embedding deal with misspelled words

Learning Resources

Lecture Slides and resources including Hybrid work (week 2-4).

Part 1: Evaluating semantic similarity With Human Judgments

Objective

In this task, you will evaluate how well different pre-trained word embedding models capture human-judged semantic similarity, using the SimLex-999 benchmark data.

Overview

SimLex-999 dataset is a human-annotated dataset that measures the semantic similarity of word pairs. It is a gold-standard dataset designed to measure true semantic similarity.

By comparing embedding-based similarity scores with SimLex-999 ratings, we can evaluate the performance of different word embedding with intrinsic evaluation measure(cosine similarity).

Dataset (Provided on Brightspace)

SimLex-999 is a gold-standard dataset used to evaluate models that learn word meanings and concepts. It provides similarity scores for 999 word pairs, with the following characteristics:

- **Semantic Similarity**: Measures how similar the meanings of two words are, not just their relatedness or association.
- **Tab-separated file**: Each row represents a word pair, and columns provide details about the pair.

Key Columns in SimLex-999:

Column	Description
word1	The first word in the pair.
word2	The second word in the pair.
POS	The part-of-speech (noun, verb, adjective) shared by the word pair.
SimLex999	Human-annotated similarity score (0 to 10). Higher values indicate greater semantic similarity.
conc(w1)	Concreteness rating (1-7) for word1.
conc(w2)	Concreteness rating (1-7) for word2.
concQ	Quartile based on concreteness ratings.
Assoc(USF)	Strength of free association between word1 and word2.
SimAssoc333	Binary indicator for the 333 most associated pairs.
SD(SimLex)	Standard deviation of annotator scores for the similarity rating.

Instructions

Step 1: Explore your data

- Download the SimLex-999 dataset from Brightspace.
- From the data file, Select the three columns **word1**, **word1** and **SimLex999**
- Compute and display the minimum, average, and maximum similarity values from SimLex999 similarity column.

Step 2: Select highly similar word pairs

Sort the dataset by the SimLex999 similarity score in descending order then select the top 60-word pairs with the highest similarity scores.

Step 3: Load two pre-trained word embedding models

- **Word2Vec** (word2vec-google-news-300).
- **GloVe** (glove-wiki-gigaword-300).

Step 4: Computing Embedding-Based Similarity

For each word pair in the selected top 60, compute the similarity scores using:

A. Word2Vec embeddings.
B. GloVe embeddings.

Step 5: Results format

Create a table to display the results with the following columns:

| word1 | word2 | similarity_w2v | similarity_ glove | similarity SimLex999|

Step 6: Analysis and Discussion

Briefly analyze which embedding model better aligns with SimLex-999 scores for the top-60 pairs, highlight examples where embeddings underestimate human similarity, compare Word2Vec and GloVe behavior.

Part 2: Word Embedding Similarity Using FastText

Objective

- Understand the importance of sub word modeling in NLP.
- discover a very cool property of word embeddings through analogies.

Overview

FastText that has been trained on a part of the Google News dataset (about 100 billion words). The model variant that we will use, contains 300-dimensional vectors for 3 million words and phrases. word embeddings that have been pre-trained trained on a combination of **Wikipedia** articles and **news datasets**. FastText not only captures the meaning of entire words but also considers sub word information (e.g., character n-grams), which allows it to better handle rare or misspelled words.

Step 1: load the pre-trained FastText

You should load the pre-trained version FastText **cc.en.300.bin**

Step 2: Check word analogies

Use the pre-trained Fasttext embeddings to implement the following analogies.

```
King — Man + Woman  = …//print the output here
computer programmer – man + woman  = …//print the output here
Doctor - Man + Woman  = …//print the output here
career – man + woman = …//print the output here
intelligent - scientist + woman …//print the output here
```

Step 3: Results and Discussion

Print your output using the previous format and write your observation about the results.

Step 4: Comparing Word2Vec and FastText for Handling Misspellings

- Create the following test word list of correctly spelled words and their misspelled variants:

```
test_words = { "correct": ["apple", "banana",
"computer", "science", "education"],
"misspelled": ["appple", "bananna", "computar",
"sciience", "edcation"] }
```

- Write a function to calculate the similarity score between the correct word and the misspelled word using cosine similarity. If either word is missing from the vocabulary, return None to indicate that the word cannot be processed.

Step 5: Results format

Your output should be formatted Like the following example:

```
Correct: apple, Misspelled: appple
Word2Vec Similarity: ……//Print the value here
FastText Similarity: ……//print the value here

.....
```

Submission Instruction

Instructions for Adding Headers in Jupyter Notebook Code

For better organization and readability of your code, please include **clear headers** at the beginning of each task or section in your Jupyter notebook. These headers should:

1. **Describe the Task**: Clearly state the task or step the code is performing.
2. **Use Markdown Cells**: Use Markdown cells (not code cells) to create headers, so they stand out visually.
3. **Format the Header**: Use appropriate Markdown syntax for headers (e.g., # for main titles, ## for subheadings).

Submission Instruction

Submit your code **as a Jupyter Notebook with the running code**, you can do the following steps:

1. Open your Jupyter Notebook: You can open Jupyter Notebook either from the command line by typing "jupyter notebook" or from Anaconda Navigator.
2. Write your code: Write the code you want to submit in the cells of the Jupyter Notebook. **Make sure to run the cells so that the output is generated**.
3. Save the Notebook: Once you have written and run your code, save the Jupyter Notebook by clicking on File -> Save and Checkpoint or by pressing "Ctrl + S".
4. Export the Notebook: To export the Jupyter Notebook, go to File -> Download as -> Notebook (.ipynb). This will download the Notebook as a .ipynb file on your laptop

Important Information

For your assignment submission, follow these steps:

1. Create a new folder named lab3
2. Place the following files inside the lab3 folder:
    - Lab3_part1
    - Lab3_part2
3. Compress the entire lab2 folder into a zip file

Submit your program via Brightspace. Your program file must be named “lab3” You can submit multiple times, with only the most recent submission (before the due date) graded.

Due Date

Check Brightspace for due dates.

Grading Criteria (Part 1)

Criteria	Points
Exploring data and select highly similar word pairs	10
Computing Embedding-Based Similarity	20
Results table	5
Analysis & discussion	5

Grading Criteria (Part 2)

Criteria	Points
Word analogies	10
Handling Misspellings	15
Results format	5